Multi-Source Candidate Data Transformer — Technical Design

Rama Lokesh Reddy Penumallu · rlpreddy565@gmail.com · Eightfold Engineering Intern Assignment · github.com/ramalokeshreddyp/Candidate-Data-Transformer

Pipeline / Step Breakdown

extract (per-source, into a flat FieldValue list) → normalize → merge (conflict resolution + confidence) → build canonical profile → project (apply runtime config) → validate. Extractors never write to the canonical profile directly — each emits FieldValue records tagged with field, source, method, and confidence, so every parser is testable in isolation and the merge engine never needs to know a CSV layout or an API's response shape. This maps to candidate_transformer/pipeline.py orchestrating the extractors, core/merge.py, core/project.py, and core/validate.py.

Canonical Output Schema and Normalization

The canonical profile (candidate_id, full_name, emails[], phones[], location, links, headline, years_experience, skills[], experience[], education[], provenance[], overall_confidence) is the source of truth and stays stable regardless of what a client requests. **Phones** normalize to E.164, returning null under ten digits rather than guessing a country code. **Dates** normalize to YYYY-MM (ISO, month-name-plus-year, or year-only inputs). **Skills** use a curated alias map ("js"/"reactjs" → "JavaScript"/"React") rather than a full taxonomy. **Country** is ISO-3166 alpha-2, inferred only when a source states it explicitly.

Merge and Conflict-Resolution Policy

Confidence is computed per field-source pair via a small prior table, not one trust score per source: a recruiter CSV is authoritative for phone/email since a human typed it deliberately, but weak for skills; a GitHub profile is the inverse — weak for "current company" but strong, if indirect, evidence for skills inferred from repository languages, discounted relative to a self-declared skill. **Single-value fields** (name, headline): highest confidence wins; exact ties break by a fixed source-priority order, so identical inputs always produce identical output, with the loser kept visible in provenance. **Multi-value fields** (emails, phones): union and deduplicate — two emails from two sources are two facts, not a conflict. **Skills** group by canonical name, keeping max confidence and the contributing-sources list. **Experience** deduplicates by company+title; on collision the richer entry wins, with the loser still recorded for audit. Overall confidence is the mean of confidences belonging to values that actually won a slot, not an average across every candidate value.

Runtime Custom-Output Configuration

A projection layer (core/project.py) reads the canonical profile once and reshapes it per a JSON config supplied at runtime: selecting a subset of fields, remapping from a canonical path (mapping over a list, e.g. skill names, or indexing the first email), applying per-field normalization, and following an explicit on_missing policy — null, omit, or error. Projection never mutates the canonical record, so it is computed once and projected many times, cheaply and consistently. Validation (core/validate.py) runs immediately after; under "error", a missing required field raises with the specific field named.

Edge Cases Handled

- Malformed/garbage source (wrong columns, broken JSON): extractor catches the failure and returns empty rather than raising or inventing a value.
- A source missing entirely: pipeline still produces a valid, partial profile; absent fields are null, never guessed.
- Conflicting names/titles across sources: resolved deterministically by confidence then source priority, with the loser kept in the audit trail.
- Required field missing under on_missing="error": raises a specific, named projection error instead of an incomplete object.

What Was Deliberately Left Out Under Time Pressure

The implementation covers Recruiter CSV and ATS JSON (structured) plus GitHub profile and repositories (unstructured) — one source per required group. LinkedIn and resume extractors are accounted for in the schema and priors but not implemented; phone normalization is regex-based, skipping a default country code. Tests (13/13 passing) and sample output are in the repository.